

Sort Algorithms

@author Yangxin Wu

Last edited: 2025.06.11

- [Overview](#)
 - [Stability](#)
 - [Internal / External Sort](#)
- [Selection Sort](#)
 - [Naive Implementation](#)
 - [Heap Sort](#)
- [Insertion Sort](#)
 - [Naive Implementation](#)
 - [Optimize Using Binary Search](#)
 - [Optimize Using Linked List](#)
 - [Shell Sort](#)
- [Divide and Conquer](#)
 - [Merge Sort](#)
 - [Quick Sort](#)
 - [Median and Order Statistics](#)
 - [Multiplication Algorithm](#)
 - [Problem of Perfect Cover of Chessboard](#)
- [Radix Sort](#)
- [Conclusion Of Sort Algorithms](#)

✍ 写在前面：在实际算法实践中，只要会写归并排序和快速排序就够了。前者是稳定排序，后者通常是最快的，它们能够应对绝大部分的场景。

Overview

排序是计算机程序设计中的一种重要操作，它的功能是将一个数据元素的任意序列，重新排列成一个按关键字有序的序列。

Stability

稳定性是指相等的元素经过排序之后相对顺序是否发生了改变。拥有稳定性这一特性的算法会让原本有相等键值的纪录维持相对次序。

- 稳定排序：基数排序、计数排序、插入排序、冒泡排序、归并排序。
- 不稳定排序：选择排序、堆排序、快速排序、希尔排序。

Internal / External Sort

- 内部排序 (**Internal sorting**)：在排序过程中，只使用计算机的内存存放待排序记录。内部排序的过程是一个逐步扩大记录的有序序列长度的过程。我们可以根据这一朴素的思想进一步将内部排序分为下面几类：
 1. 插入类：将无序子序列中的一个或几个记录“插入”到有序序列中，从而增加记录的有序子序列的长度。
 2. 选择类：从记录的无序子序列中“选择”关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。
 3. 归并类：通过“归并”两个或两个以上的记录有序子序列，逐步增加记录有序序列的长度
 4. 交换类：通过“交换”无序序列中的记录从而得到其中关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。
- 外部排序 (**External sorting**)：排序期间文件的全部记录不能同时存放在计算机的内存中，要借助计算机的外存才能完成排序。
内外存之间的数据交换次数是影响外部排序速度的主要因素。

定义序列如下：

```
#define MAXSIZE 1000          // 待排顺序表最大长度
typedef int KeyType;          // 关键字类型为整数类型

typedef struct {
    KeyType key;               // 关键字项
    InfoType otherinfo;       // 其它数据项
} RcdType;                   // 记录类型

typedef struct {
    RcdType r[MAXSIZE+1];     // r[0]闲置
    int length;               // 顺序表长度
} SqList;                    // 顺序表类型
```

Selection Sort

选择排序 (Selection sort) 是一种简单直观的排序算法。它的工作原理是每次找出第 i 小的元素（也就 $A_{i \dots n}$ 是最小的元素），然后将这个元素与数组第 i 个位置上的元素交换。

Naive Implementation

时间复杂度：

- 比较次数： $O(n^2)$
- 最好移动次数： $O(1)$

- 最差移动次数： $O(n)$
空间复杂度： $O(n)$

代码：

```
void SelectSort(Sqlist &L) {
    for (int i=1; i<L.length; ++i) {
        int low = i;

        for(int j=i+1; j<=L.length; ++j) {
            if (L.r[j].key < L.r[low].key) {
                low = j;
            }
        }

        if (i!=low) {
            swap(L.r[i], L.r[low]);
        }
    }
}
```

Heap Sort

堆排序的本质是建立在二叉堆上的选择排序。

- 小根堆

我们把具有如下性质的数组 A 表示的完全二叉树称为小根堆：

1. 若 $2i \leq n$ ，则 $A[i].key \leq A[2i].key$
 2. 若 $2i + 1 \leq n$ ，则 $A[i].key \leq A[2i + 1].key$
- 对于任意一个非叶结点，其关键字都不大于其左、右子结点的关键字。

- 大根堆

把具有如下性质的数组 A 表示的完全二叉树称为大根堆：

1. 若 $2i \leq n$ ，则 $A[i].key \geq A[2i].key$
 2. 若 $2i + 1 \leq n$ ，则 $A[i].key \geq A[2i + 1].key$
- 对于任意一个非叶结点，其关键字都不小于其左、右子结点的关键字。

 算法思想：

1. 首先建立大根堆，然后将堆顶的元素取出，作为最大值与数组尾部的元素交换，并维持残余堆的性质；
2. 之后将堆顶的元素取出，作为次大值，与数组倒数第二位元素交换，并维持残余堆的性质；

3. 以此类推，在第 $n - 1$ 次操作后，整个数组就完成了排序。

由此可见，堆排序就是利用了大根堆的性质（对于任意一个非叶结点，其关键字都不小于其左、右子结点的关键字）后，自然而然得到的排序算法。

如何构造为一个二叉堆？

1. 构造一个函数 `heapify`，作用是维护以 `i` 为根节点的子树的最大堆性质（`i` 的键值 `arr[i]` 在该子树中最大）。
 - a. 首先比较根节点与左右子节点的值，如果子节点更大则更新 `largest` 的索引，再交换 `arr[i]` 和 `arr[largest]` 的地址。
 - b. 交换后，新的根节点有可能破坏了该子树的最大堆性质，所以需要递归地对 `largest` 所在的子树再次调用 `heapify`。
2. 现在正式建堆。我们从最后一个非叶子节点（注意到其在数组中的索引是 $n / 2 - 1$ ）开始，向上遍历至根节点，过程中对每个节点都执行 `heapify`。这样可以确保每个子树都满足最大堆的性质，最终整个数组都会变成一个最大堆。

以大根堆为例，构造代码如下：

```
/**
 * @brief 维护最大堆性质的函数
 * @param arr 数组
 * @param n 数组的有效长度
 * @param i 要堆化的子树的根节点索引
 */
void heapify(int arr[], int n, int i) {
    int largest = i;        // 初始化最大元素为根节点
    int left = 2 * i + 1;   // 左子节点索引
    int right = 2 * i + 2;  // 右子节点索引

    // 如果左子节点存在且大于当前最大元素
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    // 如果右子节点存在且大于当前最大元素
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // 如果最大元素不是根节点，则交换并递归地堆化受影响的子树
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

/**
 * @brief 构建最大堆的函数
```

```

* @param arr 数组
* @param n 数组的长度
*/
void buildMaxHeap(int arr[], int n) {
    // 最后一个非叶子节点的索引是 (n/2 - 1)
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

```

时间复杂度： $O(n \log_2 n)$ （建堆： $O(n \log_2 n)$ ，排序 + 维护： $O(n \log_2 n)$ ）

空间复杂度： $O(1)$

堆排序的示例代码如下：

```

/**
 * @brief 堆排序函数
 * @param arr 待排序的数组
 * @param n 数组的长度
 */
void heapSort(int arr[], int n) {
    // 构建最大堆
    buildMaxHeap(arr, n);

    // 一个一个地从堆中提取元素
    for (int i = n - 1; i > 0; i--) {
        // 将当前堆顶元素与当前数组的最后一个元素交换
        swap(&arr[0], &arr[i]);
        // 对剩余的堆进行堆化
        heapify(arr, i, 0); // i 是当前堆的有效长度
    }
}

```

Insertion Sort

插入排序（Insertion sort）是一种简单直观的排序算法。

工作原理为将待排列元素划分为「已排序」和「未排序」两部分，每次从「未排序的」元素中选择一个插入到「已排序的」元素中的正确位置。

类似于打扑克牌时，从牌桌上抓一张牌，按牌面大小插到手牌后，再抓下一张牌。

Naive Implementation

时间复杂度：

- 最好情况：比较 $O(n)$ ，移动 $O(1)$
- 最坏情况：比较 $O(n^2)$ ，移动 $O(n^2)$

- 平均: $O(n^2)$
空间复杂度: $O(1)$

代码:

```
void InsertionSort(SqList &L) {
    for (int i = 2; i <= L.length; ++i) {
        if (L.r[i].key < L.r[i-1].key) {
            L.r[0] = L.r[i];

            for (int j = i-1; L.r[0].key < L.r[j].key; --j) {
                L.r[j+1] = L.r[j];
            }

            L.r[j+1] = L.r[0];
        }
    }
}
```

Optimize Using Binary Search

由于“每次从「未排序的」元素中选择一个插入到「已排序的」元素中”本身是通过顺序查找来找到目标元素应该插入的位置，我们很自然地想到可以将顺序查找替换为更优的查找算法，也就是这里要用的二分查找。

使用二分查找在某一数组中查找元素的前提条件是该数组是有序的。

这里简单讲讲二分查找的思想：

1. 每次考察数组当前部分的中间元素，如果中间元素刚好是要找的，就结束搜索过程；
2. 如果中间元素小于所查找的值，那么左侧的只会更小，不会有所查找的元素，只需到右侧查找；
3. 如果中间元素大于所查找的值同理，只需到左侧查找。

二分查找的最优时间复杂度为 $O(1)$ ，平均时间复杂度和最坏时间复杂度均为 $O(\log n)$ 。

使用二分查找改进后的插入排序如下：

```
int BinarySearch(SqList &L) {
    int low = 1;
    int high = i-1;

    while (low <= high) {
        int m = (low + high) / 2;

        if (L.r[0].key < L.r[m].key) {
            high = m-1;
        } else {
```

```

        low = m+1;
    }
}
return high;
}

void BSInsertionSort(Sqlist &L) {
    for(int i = 2; i <= L.length; ++i) {
        L.r[0] = L.r[i];

        int high = BinarySearch(&L)

        for(int j = i-1; j >= high+1; --j) {
            L.r[j+1] = L.r[j];
        }

        L.r[high+1] = L.r[0];
    }
}

```

Optimize Using Linked List

我们不难发现在插入过程中，由于数组结构的局限性，我们只能将原数组中的数一位一位的向右移动，再将待插入的数放入产生的空位中。这也启发我们改进数组这一数据结构，引入在插入、删除操作中具有极大优势的链表（**Linked List**）结构。

基本思想：

1. 构建一个结构体类型，使其封装了值域和指针域。
2. 初始化静态链表：插入第一个结点。由于链表中只有一个元素，当然是有序的。
3. 把后续的结点依次插入到该循环链表中，调整各结点的指针指向使其沿着指针方向是有序的。

链表类的构造如下：

```

#define SIZE 100

typedef struct {
    RcdType rc;           //记录项
    int next;             //指针项
}SLNode;                 //表结点类型

typedef struct {
    SLNode r[SIZE];       //0号单元为表头结点
    int length;           //链表当前长度
}SLinkListType;

/**

```

```

* @brief 初始化静态链表
* 设置0号单元为头结点，其next指向第一个数据结点，
* 并构建空闲链表。
* @param L 指向静态链表结构体的指针
*/
void InitSLinkList(SLinkListType *L) {
    L->length = 0;

    L->r[0].next = 0;
    for (int i = 1; i < SIZE - 1; i++) {
        L->r[i].next = i + 1;
    }
    L->r[SIZE-1].next = 0;

    L->free_head = 1;
}

/**
* @brief 从空闲链表中获取一个新节点的索引
* @param L 指向静态链表结构体的指针
* @return 新节点的索引，如果空闲链表为空则返回0
*/
int MallocSLL(SLinkListType *L) {
    if (L->free_head == 0) {
        printf("Error: Static Link List is full!\n");
        return 0;
    }
    int newNodeIdx = L->free_head;
    L->free_head = L->r[newNodeIdx].next;
    return newNodeIdx;
}

/**
* @brief 将节点归还到空闲链表
* @param L 指向静态链表结构体的指针
* @param idx 要归还的节点的索引
*/
void FreeSLL(SLinkListType *L, int idx) {
    if (idx > 0 && idx < SIZE) {
        L->r[idx].next = L->free_head;
        L->free_head = idx;
    }
}

```

优化后的插入排序如下：

```

/**
* @brief 将一个值有序地插入到静态链表中（插入排序的核心）
* @param L 指向静态链表结构体的指针

```



```

* @param value 要插入的值
* @return 1表示成功，0表示失败（空间不足）
*/
int InsertOrderedSLinkList(SLinkListType *L, RcdType value) {
    int newNodeIdx = MallocSLL(L);
    if (newNodeIdx == 0) {
        return 0;
    }
    L->r[newNodeIdx].rc = value; ``
    int pre = 0;
    int cur = L->r[0].next;
    while (cur != 0 && L->r[cur].rc < value) {
        pre = cur;
        cur = L->r[cur].next;
    }
    L->r[newNodeIdx].next = cur;
    L->r[pre].next = newNodeIdx;
    L->length++;
    return 1;
}

```

△但是，链表不仅带来了插入和删除的优势，也使得取值即为繁琐。由于该数据结构的限制，我们想要得到其中的某一个元素必须从头节点开始遍历，直到找到目标节点，而不能像数组那样根据索引随机取值。

Shell Sort

希尔排序（Shell sort），也称为缩小增量排序法，是插入排序的一种改进版本。希尔排序以它的发明者 Donald Shell 命名。

排序对不相邻的记录进行比较和移动：

1. 将待排序序列分为若干子序列（每个子序列的元素在原始数组中间距相同）；
2. 对这些子序列进行插入排序，将数组变为“d 有序”，也就是数组中由每相隔 d 个位置的数构成的子数组是有序的；
3. 减小每个子序列中元素之间的间距，重复上述过程直至间距减少为 1。

例如：将 n 个记录分成 d 个子序列：

$$\begin{aligned}
 &\{R[1], R[1+d], R[1+2d], \dots, R[1+kd]\} \\
 &\{R[2], R[2+d], R[2+2d], \dots, R[2+kd]\} \\
 &\quad \dots\dots \\
 &\{R[d], R[2d], R[3d], \dots, R[kd], R[(k+1)d]\}
 \end{aligned}$$

其中， d 称为增量，它的值在排序过程中从大到小逐渐缩小，直至最后一趟排序减为 1。

希尔排序是目前为止唯一无法准确描述其对于乱序的数组的排序性能的排序方法。

时间复杂度：

- 最好情况： $O(n \log n)$
- 平均 $O(n^{1.3})$ 到平均 $O(n^{1.5})$
 希尔排序的平均时间复杂度和最坏时间复杂度与间距序列的选取有关。
 设间距序列为 H ，下面给出 H 的两种经典选取方式，这两种选取方式均使得希尔排序算法的复杂度降为 $o(n^2)$ 级别。（[OI Wiki](#) 有即为详尽的证明，考试不要求）
- 命题 1: 若间距序列为 $H = \{2^k - 1 \mid k = 1, 2, \dots, \lfloor \log_2 n \rfloor\}$ （从大到小），则希尔排序算法的时间复杂度为 $O(n^{3/2})$ 。
- 命题 2: 若间距序列为 $H = \{k = 2^p \cdot 3^q \mid p, q \in \mathbb{N}, k \leq n\}$ （从大到小），则希尔排序算法的时间复杂度为 $O(n \log^2 n)$ 。
 空间复杂度： $O(1)$

示例代码如下：

```
void ShellInsert(SqList &L, int dk) {
    for (int i = dk+1; i <= L.length; ++i) {
        if (L.r[i].key < L.r[i-dk].key) {
            L.r[0] = L.r[i]; // 暂存在R[0]

            for(int j=i-dk; j>0 && (L.r[0].key < L.r[j].key); j-=dk) {
                L.r[j+dk] = L.r[j]; // 记录后移，查找插入位置
            }

            L.r[j+dk] = L.r[0]; // 插入
        }
    }
}

void ShellSort(SqList &L, int delta[], int t) {
    // 增量为delta[]的希尔排序，t是delta的长度
    for(int k = 0; k < t; ++k) {
        ShellInsert(L, delta[k]); // 一趟增量为delta[k]的插入排序
    }
}
```

Divide and Conquer

分治（Divide and Conquer），字面上的解释是「分而治之」，就是把一个复杂的问题分成两个或更多的相同或相似的子问题，直到最后子问题可以简单的直接求解，原问题的解即子问题的解的合并。

大概的流程可以分为三步：分解 → 解决 → 合并。

1. 分解原问题为结构相同的子问题。
2. 分解到某个容易求解的边界之后，进行递归求解。
3. 将子问题的解合并成原问题的解。

分治法能解决的问题一般有如下特征：

1. 该问题的规模缩小到一定的程度就可以容易地解决。
2. 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质，利用该问题分解出的子问题的解可以合并为该问题的解。
3. 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

△ 如果各子问题是不独立的，则分治法要重复地解公共的子问题，也就做了许多不必要的工作。此时虽然也可用分治法，但一般用动态规划较好。

两个优秀的基于分治思想的排序算法：

- 归并排序：选择一个位置将序列划分成两个部分。
- 快速排序：选择一个枢轴 x ，根据元素与 x 的大小关系来划分。

Merge Sort

归并排序（Merge Sort）基于分治思想将数组分段排序后合并。

时间复杂度在最优、最坏与平均情况下均为 $\Theta(n \log n)$ ，空间复杂度为 $\Theta(n)$ 。

归并排序可以只使用 $\Theta(1)$ 的辅助空间，但为便捷通常使用与原数组等长的辅助数组。

 算法思想：

1. 当数组长度为 1 时，该数组就已经是有序的，不用再分解。
 2. 当数组长度大于 1 时，该数组很可能不是有序的。此时将该数组分为两段（为了保证算法复杂度一般是二等分），再分别检查两个数组是否有序。如果有序，则将它们合并为一个有序数组；否则对不有序的数组重复第 2 条，再合并。
- 用数学归纳法可以证明该流程可以将一个数组转变为有序数组。

合并过程（将两个有序的数组 $a[i]$ 和 $b[j]$ 合并为一个有序数组 $c[k]$ ）：

1. 从左往右枚举 $a[i]$ 和 $b[j]$ ，找出最小的值并放入数组 $c[k]$ ；
 2. 重复上述过程直到 $a[i]$ 和 $b[j]$ 有一个为空时，将另一个数组剩下的元素放入 $c[k]$ 。
- 为保证排序的稳定性，前段首元素小于或等于后段首元素时（ $a[i] \leq b[j]$ ）而非小于时（ $a[i] < b[j]$ ）就要作为最小值放入 $c[k]$ 。

示例代码如下：

```
/**
 * @brief 合并两个已排序的子数组
 * @param arr 原始数组
 * @param left 左子数组的起始索引
 * @param mid 左子数组的结束索引 / 右子数组的起始索引-1
 * @param right 右子数组的结束索引
 */
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
```

```

int n1 = mid - left + 1; // 左子数组的长度
int n2 = right - mid; // 右子数组的长度

// 创建临时数组来存储左右子数组
int *L = (int *)malloc(n1 * sizeof(int));
int *R = (int *)malloc(n2 * sizeof(int));
if (L == NULL || R == NULL) {
    fprintf(stderr, "内存分配失败!\n"); exit(EXIT_FAILURE);
}

// 将数据复制到临时数组 L[] 和 R[]
for (i = 0; i < n1; i++) {
    L[i] = arr[left + i];
}

for (j = 0; j < n2; j++) {
    R[j] = arr[mid + 1 + j];
}

// 合并临时数组回到 arr[left...right]
i = 0; // 左子数组的初始索引
j = 0; // 右子数组的初始索引
k = left; // 合并后的数组的初始索引
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

// 复制 L[] 中剩余的元素（如果有的话）
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// 复制 R[] 中剩余的元素（如果有的话）
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

// 释放临时数组的内存
free(L);

```

```

    free(R);
}

/**
 * @brief 归并排序主函数
 * @param arr 待排序的数组
 * @param left 数组的起始索引
 * @param right 数组的结束索引
 */
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        // 找到中间点
        int mid = left + (right - left) / 2; // 避免 left+right 溢出
        // 递归地对左右两半进行排序
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        // 合并已排序的两半
        merge(arr, left, mid, right);
    }
}

```

Quick Sort

我们先简单介绍一下冒泡排序（Bubble Sort）：每次检查相邻两个元素，如果前面的元素与后面的元素满足给定的排序条件，就将相邻两个元素交换。当没有相邻的元素需要交换时，排序就完成了。经过 i 次扫描后，数列的末尾 i 项必然是最大的 i 项，因此冒泡排序最多需要扫描 $n-1$ 遍数组就能完成排序。

在序列完全有序时，冒泡排序只需遍历一遍数组，不用执行任何交换操作，时间复杂度为 $O(n)$ 。

在最坏情况下，冒泡排序要执行 $\frac{(n-1)n}{2}$ 次交换操作，时间复杂度为 $O(n^2)$ 。

冒泡排序的平均时间复杂度为 $O(n^2)$ 。

示例代码如下：

```

void bubble_sort(int *a, int n) {
    bool flag = true;
    // 没有发生交换后跳出循环
    while (flag) {
        flag = false;
        for (int i = 1; i < n; ++i) {
            if (a[i] > a[i + 1]) {
                flag = true; // 发生了交换
                int t = a[i]; // 交换
                a[i] = a[i + 1];
                a[i + 1] = t;
            }
        }
    }
}

```

```
}  
}
```

快速排序（Quick Sort）是冒泡排序的改进版，也是最好的一种内排序，在很多面试题中都会出现，也是作为程序员必须掌握的一种排序方法。

 算法思想：

1. 在待排序的元素任取一个元素作为基准（Pivot，通常选第一个元素，但最的选择方法是从待排序元素中随机选取一个作为基准），称为基准元素；
2. 将待排序的元素进行分区，比基准元素大的元素放在它的右边，比其小的放在它的左边；
3. 对左右两个分区重复以上步骤直到所有元素都是有序的。

复杂度分析：

- 最优情况：每一次选择的分界值都是序列的中位数，此时算法时间复杂度满足的递推式为 $T(n) = 2T(\frac{n}{2}) + \Theta(n)$ ，可得 $T(n) = \Theta(n \log n)$ 。故最优时间复杂度为 $O(n \log n)$ 。
- 最坏情况：每一次选择的分界值都是序列的最值，此时算法时间复杂度满足的递推式为 $T(n) = T(n-1) + \Theta(n)$ ，累加可得 $T(n) = \Theta(n^2)$ 。故最坏时间复杂度为 $O(n^2)$ 。
- 平均情况，每一次选择的分界值可以看作是等概率随机的，时间复杂度为 $O(n \log n)$ 。

Proof:

Lemma 1: 当对 N 个元素的数组进行快速排序时，假设在划分元素时总共的比较次数为 X ，则快速排序的时间复杂度是 $O(N + X)$ 。

Proof 1:

由于在每次划分元素的过程中，都会选择一个元素作为分界值。而在最坏情况下，每次划分只能确定一个元素的位置，所以划分元素的过程至多发生 N 次。

在每一次划分过程中，我们都需要将当前子数组中的元素与分界值进行比较，次数与当前子数组的元素数量相等。总的比较次数 X 是所有划分过程中比较次数的总和。

由上可知，快速排序的总时间复杂度是 $O(N + X)$ 。

设 a_i 为原数组中第 i 小的数。

定义 p_{ij} 为在排序过程中 a_i 是否和 a_j 发生比较的概率。

定义 X_{ij} 服从成功概率为 p_{ij} 的伯努利分布，即 $X_{ij} \sim \text{Bernoulli}(p_{ij})$ 。

其中 $1 \leq i \leq n-1, i \leq j \leq n$ 。

我们不难得出，总的比较次数 X 可以表示为所有可能存在的 X_{ij} 之和：

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

为了得到快速排序的平均比较次数，我们计算 X 的期望：

$$\begin{aligned}
E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^n p_{ij}
\end{aligned}$$

Lemma 2: a_i 和 a_j 比较的充要条件是 a_i 或 a_j 是集合 A_{ij} 中第一个被选中的分界值，其中 A_{ij} 指的是包含 a_i, a_j 以及它们之间所有元素的集合 $\{a_i, a_{i+1}, \dots, a_j\}$ 。这些元素在排序后的数组中是连续的。

Proof 2:

先证必要性，即若 a_i 和 a_j 比较，则 a_i 或 a_j 是集合 A_{ij} 中第一个被选中的分界值。

我们证明其逆否命题，即若 a_i 和 a_j 都不是集合 A_{ij} 中第一个被选中的分界值，则 a_i 不和 a_j 比较。

若 a_i 和 a_j 都不是集合 A_{ij} 中第一个被选中的分界值，则一定存在一个 x 满足 $i < x < j$ ，使得 a_x 是 A_{ij} 中第一个被选中的分界值。

在以 a_x 为分界值的划分中，由于 $a_i < a_x < a_j$ ， a_i 会被划分到 a_x 的左侧子序列，而 a_j 会被划分到 a_x 的右侧子序列，所以之后 a_i 和 a_j 一定不会相互比较。

又因为元素只和分界值进行比较，在 a_x 被选为分界值之前， a_i 和 a_j 所在的子序列中没有其他元素被选为分界值（如果有就不会出现在这个待排子序列），所以它们也不会在之前或当前划分中发生比较。由上可知， a_i 不和 a_j 比较。

再证充分性，即若 a_i 或 a_j 是集合 A_{ij} 中第一个被选中的分界值，则 a_i 和 a_j 比较。

假设 a_k 是集合 A_{ij} 中第一个被选中的分界值，要么是 a_i ，要么是 a_j 。在 a_k 被选中之前，由于 A_{ij} 中没有其他数值作为分界值将他们分开，所以 A_{ij} 中的元素都在同一子序列中。在以 a_k 为分界值的划分中， a_i 或 a_j 和当前子序列中所有元素都进行了比较，所以 a_i 和 a_j 进行了比较。

引理得证。

我们现在考虑计算 p_{ij} 。在 A_{ij} 中某个元素被选为分界值之前， A_{ij} 中的元素都在同一子序列中。所以 A_{ij} 中每个元素都会等可能地被第一个被选为分界值。由于 A_{ij} 中有 $j - i + 1$ 个元素。由引理 2 可得：

$$p_{ij} = P(a_i \text{ 或 } a_j \text{ 是集合 } A_{ij} \text{ 中第一个被选中的分界值}) = \frac{2}{j - i + 1}$$

代入前面得到的期望表达式并化简可得：

$$\begin{aligned}
E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\
&= \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k} \\
&= \sum_{k=2}^n \sum_{i=1}^{n-k+1} \frac{2}{k} \\
&= \sum_{k=2}^n (n-k+1) \frac{2}{k} \\
&= 2 \sum_{k=2}^n \left(\frac{n+1}{k} - 1 \right) \\
&= 2 \left((n+1) \sum_{k=2}^n \frac{1}{k} - \sum_{k=2}^n 1 \right) \\
&= 2((n+1)(H_n - 1) - (n-1)) \\
&= O(n \log n)
\end{aligned}$$

其中， H_n 表示调和级数的前 n 项和，有 $H_n = O(\log n)$ 。

由此，快速排序的期望时间复杂度为 $O(n \log n)$ 。

空间复杂度（递归深度）

- 最好情况： $O \log n$
- 最坏情况： $O(n)$
- 平均情况： $O(\log_2 n)$

```

/**
 * @brief 快速排序
 * @param arr 原始数组
 * @param _left 初始代排区间的左端点
 * @param _right 初始代排区间的右端点
 */
void quickSort(int arr[], int _left, int _right) {
    int left = _left;
    int right = _right;
    int temp = 0;

    if (left <= right) {
        temp = arr[left]; //待排序的元素至少有两个的情况
        while (left != right) { //待排序的第一个元素作为基准元素
            //从左右两边交替扫描，直到left =
            right
            while (right > left && arr[right] >= temp) {
                //从右往左扫描，找到第一个比基准元素小
                right--;
            }
            //找到这种元素arr[right]后与
            arr[left] = arr[right];
            arr[left]交换
            while(left < right && arr[left] <= temp) {

```



```

        left++; //从左往右扫描，找到第一个比基准元素大
的元素
    }
    arr[right] = arr[left]; //找到这种元素arr[left]后，与
arr[right]交换
}
arr[right] = temp; //基准元素归位
quickSort(arr, _left, left-1); //对基准元素左边的元素进行递归排序
quickSort(arr, right+1, _right); //对基准元素右边的进行递归排序
}
}

```

Median and Order Statistics

在一个由 n 个元素组成的集合中， i 阶顺序统计量（**order statistic**）是该集合中第 i 小的元素。例如，在一个 n 元素集合中，最小值是第 1 阶顺序统计量，而最大值是第 n 个顺序统计量。

用非形式化的描述来说，一个中位数（**median**）是它所属集合的“中点元素”。当 n 为奇数时，中位数是唯一的，位于 $i = (n + 1)/2$ 处。当 n 为偶数时，存在两个中位数，分别位于 $i = n/2$ 和 $i = n/2 + 1$ 处。因此，如果不考虑 n 的奇偶性，中位数总是出现在 $i = \lfloor (n + 1)/2 \rfloor$ （下中位数）和 $i = \lceil (n + 2)/2 \rceil$ （上中位数）。为了简便起见，之后出现的“中位数”都是指下中位数。

本部分将讨论一个由 n 个互异的元素构成的集合中选择第 i 个顺序统计量的问题。为了方便起见，假设集合中的元素都是互异的。我们将这一问题形式化定义为如下的选择问题：

- 输入：一个包含 n 个（互异的）数的集合 A 和一个整数 i ， $1 \leq i \leq n$ 。
- 输出：元素 $x \in A$ ，且 x 恰好有 $i - 1$ 个其他元素小于它。

我们可以在 $O(n \log n)$ 时间内解决这个选择问题，因为我们可以用排序算法对输入数据进行排序，然后在输出数组中根据下标找出目标元素即可。本部分将介绍一些更快的算法。

- 求最大值/最小值

求一个数组的最值很简单，我们先初始化一个变量并赋值为第一个元素，然后从第二个元素开始对这个数组进行遍历，遇到更小的或者更大的后对变量进行更新即可，也就是需要经过 $n - 1$ 次比较。

如果我们需要同时获得最大值和最小值，你可能想我们只要同时维护最大值和最小值变量就可以了，也就是说我们需要 $2n - 2$ 次比较。实际上我们可以进行优化。

首先，我们同样要初始化最大值和最小值变量。如果 n 是奇数，我们就将最小值和最大值的初值都设为第一个元素的值。如果 n 是偶数，就对前两个元素做一次比较，以决定最小值和最大值的初值。然后我们成对的输入数据，将一对输入元素相互进行比较，然后把较小的与当前最小值比较，把较大的与当前最大值进行比较。这样，对每两个元素共需 3 次比较。

下面来分析一下总的比较次数。如果 n 是奇数，那么总共进行 $3\lfloor n/2 \rfloor$ 次比较。如果 n 是偶数，则是先行进行一次初始比较，然后进行 $3(n - 2)/2$ 次比较，共 $3n/2 - 2$ 次比较。因此，不管哪一种情况，总的比较次数至多是 $3\lfloor n/2 \rfloor$ 。

- 求中位数

问题定义:

输入: 由 n 个数构成的集合 X , 整数 $i = \lfloor \frac{n+1}{2} \rfloor$

输出: $x \in X$ s.t. $-1 \leq |\{y \in X | y < x\}| - |\{y \in X | y > x\}| \leq 1$

利用分治法求解:

1. 分组, 每组 5 个数, 最后一组可能少于 5 个数
2. 将每组数分别用插入排序, 选出每组元素的中位数
3. 递归调用算法求得这些中位数的中位数 MoM
4. 用 MoM 完成划分
5. 递归: 设 x 是中位数的 MoM, 划分完成后 x 的下标为 k
 - 如果 $i = k$, 则返回 x
 - 如果 $i < k$, 则在左侧子数组中递归选取第 i 大的数
 - 如果 $i > k$, 则在右侧子数组中递归选取第 $i - k$ 大的数

示例代码如下:

```
/**
 * @brief 划分函数, 返回 pivotElement 在划分后的最终位置
 * @param arr 原始数组
 * @param left 区间的左端点
 * @param right 区间的右端点
 * @param pivotElement 对 arr[left...right] 进行划分
 */
int partition(int arr[], int left, int right, int pivotElement) {
    // 找到 pivotElement 在数组中的位置
    int i;
    for (i = left; i <= right; i++) {
        if (arr[i] == pivotElement) {
            break;
        }
    }
    // 将其换到 right 位置
    swap(&arr[i], &arr[right]);

    int storeIndex = left;
    for (i = left; i < right; i++) {
        if (arr[i] < pivotElement) {
            swap(&arr[storeIndex], &arr[i]);
            storeIndex++;
        }
    }
    // 将 pivotElement 放到最终位置
    swap(&arr[right], &arr[storeIndex]);
    return storeIndex;
}
```

```

/**
 * @brief 查找 arr[left...right] 中的 k 阶统计量
 * @param arr 原始数组
 * @param left 区间的左端点
 * @param right 区间的右端点
 * @param k 在 0 到 (right - left) 之间
 */
int selectKthSmallest(int arr[], int left, int right, int k) {
    // 区间长度
    int n = right - left + 1;

    // 如果数组很小，直接排序并返回
    if (n <= 5) {
        insertionSort(arr, left, right);
        return arr[left + k];
    }

    // 分组并找到每组的中位数
    // 计算小组数量
    int numGroups = (n + 4) / 5;
    int* medians = (int*)malloc(numGroups * sizeof(int));

    for (int i = 0; i < numGroups; i++) {
        int groupLeft = left + i * 5;
        int groupRight = groupLeft + 4;
        if (groupRight > right) {
            groupRight = right;           // 最后一组可能不足5个
        }
        // 对小组进行排序
        insertionSort(arr, groupLeft, groupRight);
        // 提取小组中位数
        medians[i] = arr[groupLeft + (groupRight - groupLeft) / 2];
    }

    // 递归查找中位数的中位数
    int mom = selectKthSmallest(medians, 0, numGroups - 1, numGroups / 2);

    // 释放 medians 数组
    free(medians);

    // 用 MoM 完成划分
    int pivotIndex = partition(arr, left, right, mom);

    // 计算 MoM 在当前子数组中的相对位置
    int rank = pivotIndex - left;

    // 递归查找
    if (k == rank) {
        // MoM 就是我们要找的第 k 小元素
    }
}

```

```

        return arr[pivotIndex];
    } else if (k < rank) {
        // 在左侧子数组中查找
        return selectKthSmallest(arr, left, pivotIndex - 1, k);
    } else {
        // 在右侧子数组中查找
        return selectKthSmallest(arr, pivotIndex + 1, right, k - rank -
1);
    }
}

selectKthSmallest(arr, 0, n-1, (n+1)/2-1);

```

Multiplication Algorithm

大数乘法问题（Multiplication Algorithm）：乘数相当大大，相乘的结果超出了基本类型的表示范围，这样的数不能够直接做乘法运算。

问题定义：

输入： n 位二进制整数 X 和 Y

输出： X 和 Y 的乘积

$$\begin{array}{c}
 \begin{array}{cc}
 n/2\text{位} & n/2\text{位} \\
 \boxed{A} & \boxed{B}
 \end{array} \\
 X =
 \end{array}
 \quad
 \begin{array}{c}
 \begin{array}{cc}
 n/2\text{位} & n/2\text{位} \\
 \boxed{C} & \boxed{D}
 \end{array} \\
 Y =
 \end{array}$$

$$\begin{aligned}
 XY &= (A \times 2^{n/2} + B)(C \times 2^{n/2} + D) \\
 &= AC \times 2^n + (AD + BC) \times 2^{n/2} + BD \\
 &= AC \times 2^n + ((A - B)(D - C) + AC + BD) \times 2^{\frac{n}{2}} + BD
 \end{aligned}$$

📖 算法思想：

1. 计算 A, B, C, D ;
2. 计算 $A - B$ 和 $D - C$
3. 计算 $n/2$ 位乘法 AC 、 BD 、 $(A - B)(D - C)$;
4. 计算 $(A - B)(D - C) + AC + BD$;
5. AC 左移 n 位， $(A - B)(D - C) + AC + BD$ 左移 $n/2$ 位;
6. 计算 XY

复杂度分析：

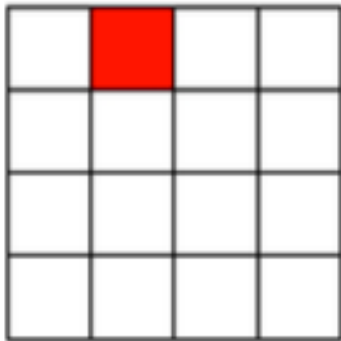
建立递归方程如下

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \end{cases}$$

时间复杂度： $T(n) = O(n^{\log_2 3}) = O(n^{1.59})$

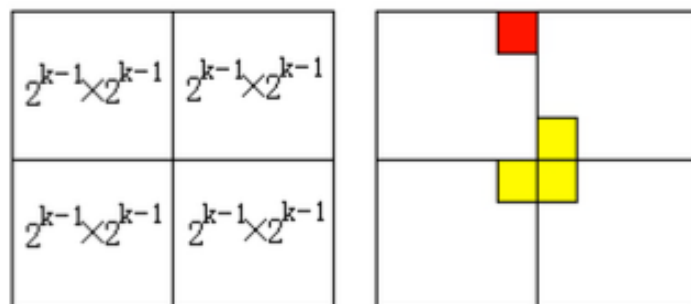
Problem of Perfect Cover of Chessboard

在一个 $2^k \times 2^k$ 个方格组成的棋盘中，有一个方格与其它的方格不同，称之为奇异块。棋盘覆盖问题，要求采用以下四种 L 型骨牌覆盖除去“奇异块”的整个棋盘，并且没有重叠。



算法思想

- 当 $k > 0$ 时，将 $2^k \times 2^k$ 的棋盘分成四块 $2^{k-1} \times 2^{k-1}$ 的子棋盘。特殊方格一定在其中的一个子棋盘中，其他相邻的三个子棋盘中没有特殊方格；
- 使用一个合适的 L 型骨牌，覆盖三个没有特殊方格的子棋盘的相邻方格。我们就得到了 4 个 $2^{k-1} \times 2^{k-1}$ 的棋盘覆盖问题；
- 继续递归处理 4 子棋盘，直到子棋盘中只剩一个特殊方格为止。



复杂度分析：
建立递归方程如下

$$T(n) = \begin{cases} \Theta(1) & \text{if } k = 1 \\ 4T(k-1) + \Theta(1) & \text{if } k > 1 \end{cases}$$

时间复杂度：

$$\begin{aligned} T(k) &= 4T(k-1) + \Theta(1) \\ &= 4(4T(k-2) + \Theta(1)) + \Theta(1) \\ &= \dots \\ &= 4^k T(0) + \Theta(1) \sum_{i=0}^{k-1} 4^i \\ &= \Theta(4^k) \end{aligned}$$

Radix Sort

基数排序（Radix Sort）是一种非比较型的排序算法，最早用于解决卡片排序的问题。基数排序将待排序的元素拆分为 k 个关键字，逐一对各个关键字排序后完成对所有元素的排序。如果是从第 1 关键字到第 k 关键字顺序进行比较，则该基数排序称为 MSD（Most Significant Digit first）基数排序；如果是从第 k 关键字到第 1 关键字顺序进行比较，则该基数排序称为 LSD（Least Significant Digit first）基数排序。

下面用 a_i 表示元素 a 的第 i 关键字。
以 MSD 基数排序为例，假如元素有 k 个关键字，对于两个元素 a 和 b ，默认的比较方法是：

- 比较两个元素的第 1 关键字 a_1 和 b_1 ，如果 $a_1 < b_1$ 则 $a < b$ ，如果 $a_1 > b_1$ 则 $a > b$ ，如果 $a_1 = b_1$ 则进行下一步；
 - 比较两个元素的第 2 关键字 a_2 和 b_2 ，如果 $a_2 < b_2$ 则 $a < b$ ，如果 $a_2 > b_2$ 则 $a > b$ ，如果 $a_2 = b_2$ 则进行下一步；
 -
 - 比较两个元素的第 k 关键字 a_k 和 b_k ，如果 $a_k < b_k$ 则 $a < b$ ，如果 $a_k > b_k$ 则 $a > b$ ，如果 $a_k = b_k$ 则 $a = b$ 。
- 可以类比字符串的比较。

Conclusion Of Sort Algorithms

内部排序方法的比较如下（Big O Notation）：

排序方法	比较次数		移动次数		稳定性	附加空间	
	最好	最差	最好	最差		最好	最差
选择排序	n^2		0	n	×	1	
堆排序	$n \log_2 n$		$n \log_2 n$		×	1	
插入排序	n	n^2	0	n^2	√	1	
归并排序	$n \log_2 n$		$n \log_2 n$		√	n	
冒泡排序	n	n^2	0	n^2	√	1	
快速排序	$n \log_2 n$	n^2	0	n^2	×	$\log_2 n$	n
基数排序	...		dn		√	n	